

SCHOOL OF COMPUTER SCIENCE AND IT YBN UNIVERSITY RANCHI

JAVA PROGRAMMING ASSIGNMENT 17

1. Access Modifiers: Public and Private

- **Description:** Create a class with two methods, one marked as `public` and the other as `private`. Attempt to access both methods from another class.
- **Example:** The `public` method should be accessible, while the `private` method should cause a compile-time error.
- **Test Case:**
 - Create a class `TestModifiers`. The class should try to access both methods from another class. Ensure that the program throws an error for the `private` method.

2. Access Modifiers: Protected

- **Description:** Write a program that uses inheritance to demonstrate how the `protected` modifier allows access to members within the same package or subclasses.
- **Example:** Create a superclass with a `protected` method and a subclass in the same package that calls this method.
- **Test Case:**
 - Define a superclass `Parent` with a `protected` method `display()`. Subclass `Child` should call `display()` from within the same package.

3. Default Access Modifier

- **Description:** Implement a class with a method that uses the default access modifier and demonstrate that it is accessible within the same package but not from another package.
- **Example:** Create two classes in different packages and test the access to the method.
- **Test Case:**
 - Define class `PackageA` in package `a` with a method using default access. Attempt to access this method from `PackageB` in package `b`.

4. Final Modifier with Variables

- **Description:** Write a program that defines a `final` variable and tries to modify it. The program should result in a compile-time error.
- **Example:** Declare a `final` integer and attempt to assign a new value to it.
- **Test Case:**
 - Define a `final int` `CONSTANT = 10;`. Attempt to reassign `CONSTANT = 20;` and observe the error.

5. Final Modifier with Methods

- **Description:** Create a class with a `final` method. Attempt to override this method in a subclass and demonstrate that this causes a compile-time error.
- **Example:** Define a superclass with a `final` method `show()` and a subclass that tries to override `show()`.
- **Test Case:**
 - Superclass `Base` contains a `final void show()`. Subclass `Derived` attempts to override `show()`, resulting in an error.

6. Final Modifier with Classes

- **Description:** Write a program that creates a `final` class and tries to inherit it in another class, which should result in a compile-time error.
- **Example:** Define a `final` class `Immutable` and try to extend it in another class.
- **Test Case:**
 - Create a `final` class `FinalClass`. Attempt to extend it in a new class `ChildClass` and observe the error.

7. Static Modifier with Variables

- **Description:** Create a program that defines a static variable and demonstrates how it is shared among all instances of a class.
- **Example:** Create a class with a static counter that increments with each new instance.
- **Test Case:**
 - Define a static `int count` in `CounterClass`. Instantiate `CounterClass` multiple times and print `count` each time.

8. Static Modifier with Methods

- **Description:** Write a program that demonstrates the use of a static method, showing how it can be accessed without creating an object of the class.
- **Example:** Define a class with a static method `printMessage()` and call it from the main method.
- **Test Case:**
 - Define a static method `static void greet()` in `GreetingClass`. Call `GreetingClass.greet()` from the main method.

9. Static Block

- **Description:** Implement a class with a static block that initializes static variables and demonstrate when this block is executed.
- **Example:** Define a class with a static block that prints a message and initialize a static variable.
- **Test Case:**
 - Define a static block in `InitializationClass` that prints "Static block executed" and initializes a static `int` value. Observe the output when the class is loaded.

10. Transient Modifier

- **Description:** Write a program to demonstrate the use of the `transient` keyword by creating a class that implements `Serializable` and uses `transient` to prevent certain variables from being serialized.
- **Example:** Define a class with a `transient` password field and serialize the object, then deserialize it and check the password field.
- **Test Case:**
 - Create a class `User` with `String username` and `transient String password`. Serialize and deserialize an instance of `User` and observe that password is null.